Final Portfolio “Study Arsenal”

Contents

Introduction

This report presents the project plan for my COMP1004 coursework, following the ADD (Architecture Design Document) framework.

The theme of my project is self-management, specifically targeted at students with the intent of organizing studies and managing time.

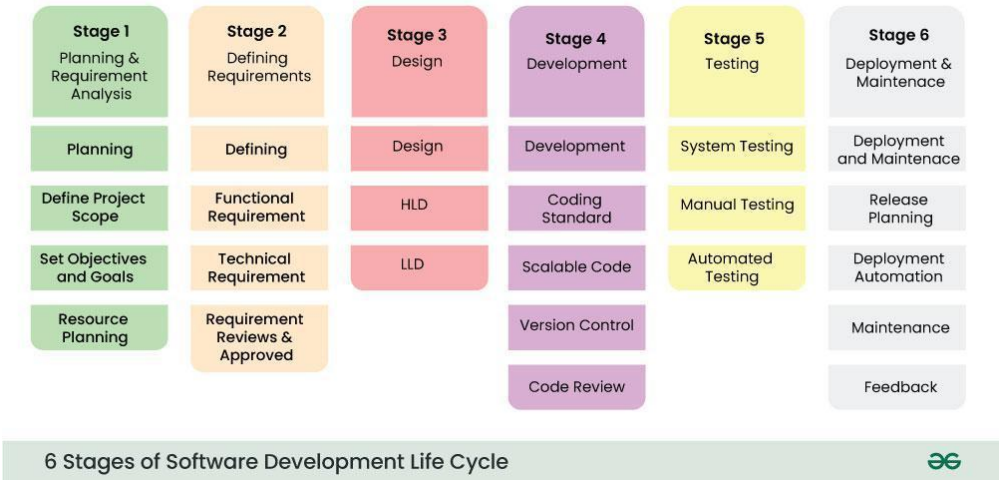
The report begins by discussing the Software Development Lifecycle (SDLC) with an agile approach, explaining its relevance and application to my project.

Next, a design documentation is presented, containing different diagrams that connect to my application, followed by an overview of the SCRUM sprints and their outcomes.

Finally, the report rounds up with a reflection on the project, reflecting on the project with a personal statement as well as highlighting challenges.

Software Development Lifecycle

Throughout my project, I followed the Software Development Lifecycle (SDLC) model, as illustrated below. This approach provided me to maintain a clear direction to follow.



Software Development Life Cycle Model SDLC Stages (GeekForGeeks, 2025)

Stage 1

My planning phase began just before the first lab session. To establish an idea, I wrote down potential ideas and their requirements. I chose the self-management theme, which I wrote down matching concepts for, such as habit tracking, study time tracking and to-do lists. Since in this time I was figuring out how to stay organized for university myself, I decided to develop an application that aligned with both my personal needs and the coursework requirements. As such, I wrote down several functions (tracking deadlines, making notes, ...) while simultaneously considering the project description (local SPA, focus on SDLC), which allowed me to set realistic goals (focus on core functions, no animations). Given the applications range of many different study-related functionalities, I named the application “Study Arsenal”.

Stage 2

In this stage, I outlined the core functionalities of my application with their requirements, which are covered later in this report under the Design Documentation section.

Stage 3

I created the initial drafts of the different required diagrams which are covered in the Design Document. These were adjusted throughout the course of the project as requirements changed, but overall, I stayed consistent with my initial idea.

Stage 4

From the first sprint to the penultimate sprint, I worked on the application using GitHub, Visual Studio Code and continual self-directed studying, as this was a student-led module. While working with version control, I was developing code on different branches and regularly pushed updates to the main branch when I finished implementing a feature. I tried to actively implement standard coding practices throughout my project, to ensure maintainability and scalability of my code:

- DRY (Don't repeat yourself) by avoiding repetitive code as implementing dynamic elements (adding tasks to a to-do list etc.) and re-using CSS classes
- YAGNI (You ain't gonna need it) by keeping the application simple and clean by not including unnecessary code
- SoC (Separation of concerns) by separating my pages and their according jQuery scripts
- OCP (Open/closed principle) by reusing existing functionalities and avoiding hardcode by using classes

Stage 5

I was testing my application all time directly by working with a live server. In addition, I used the final sprint to intentionally test all features. For this, I created a table with various test cases, provided in the Design Document.

Stage 6

The final stage is to publish the project and gain feedback. For this project, this means publishing the GitHub repository and sharing its link but also reflecting on the project by writing this documentation.

Design Document

All diagrams made in this document were done with draw.io unless stated otherwise.

Project Vision

Making the switch from high school to higher education, students often struggle with self-organisation, as studying is now conducted independently (Hockings et al., 2017). As a student myself I get to feel the importance of this by needing to keep track across various subjects which easily get scattered across various media. Therefore, the goal of my application is to provide students with an all-in-all study tracker to support their studies, with the following features:

- Keep track of tasks
- Track study time across sessions
- Add a name and notes to each session
- Get an insight into the time spent sessions
- View upcoming events

Background

It is a matter of fact that students must manage their studies on their own after proceeding into higher education. However, this skill does not come naturally, and students find themselves "feeling lost and overwhelmed by the task of planning a whole course of studies, a semester, or even an exam phase on their own" (Svartdal et al., 2020). This correlates with another study from Adams and Blair, that has found that "students would like to be organized but don't seem to have any strategies to help them do so" (2019). Therefore, it is important to provide students a clear structure to follow, which my application achieves by giving the student a fixed place to manage a variety of useful organisational tasks for their studies in.

One of the crucial factors in the success of self-directed learning is the effective usage of time, which has been "associated with greater academic performance and lower levels of anxiety in students" (Adams and Blair, 2019). This begins with a student being aware of the time they spend, as the feeling over being in control of time improves work-life balance and pressure. To support time awareness, my application provides the user a timer to track their study time and an insight on the time spend afterwards with a visual representation.

Moreover, their article showed the "scheduling and tracking of activities" (Adams and Blair, 2019) to be beneficial for time management and in lowering anxiety (Lang, 1992, cited in Adams and Blair, 2019). My application provides the user the functionality to track their tasks and deadlines, as well as their individual study sessions.

In summary, my application provides students with tools to plan their studies and improve their time-management to subsequently improve academic success and reduce anxiety.

Functional and non-functional requirements

The functional requirements contain the core features I want to implement whereas the non-functional requirements cover the performance, security and reliability standards.

Functional requirements

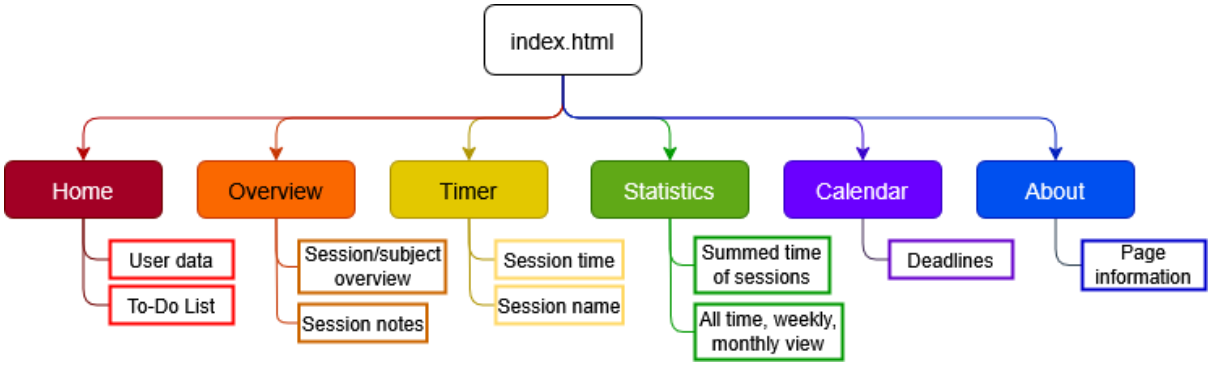
Requirement	Description
Profile management	Users can edit their name and profile picture
Task management	Users can add, delete and mark tasks as completed in their to-do list
Session notes	Users can add and view notes for recorded sessions and delete them when needed
Study timer	Users can time their study sessions in real time with an in-built timer or manually
Study statistics	Users can gain a graphic insight of their time spent per subject with a stacked bar chart
Calendar events	Users can add events with names and dates and see the countdown in days
About page	Users can view information about the app

Non-functional requirements

Requirement	Description
Performance	Navigation between pages should be smooth without full reloads (SPA)
Security	Sensitive data must not be exposed in client-side storage
Authentication	Not needed since using local storage
Scalability	Handle large JSON storage files
Data persistence	User data persists in local storage even after user reloads the page and the user can delete, export and import their data
Usability	UI should be easy to use (e.g. managing tasks and sessions)
Responsiveness	App must be scalable across different devices
Availability	App works offline since relying on local storage

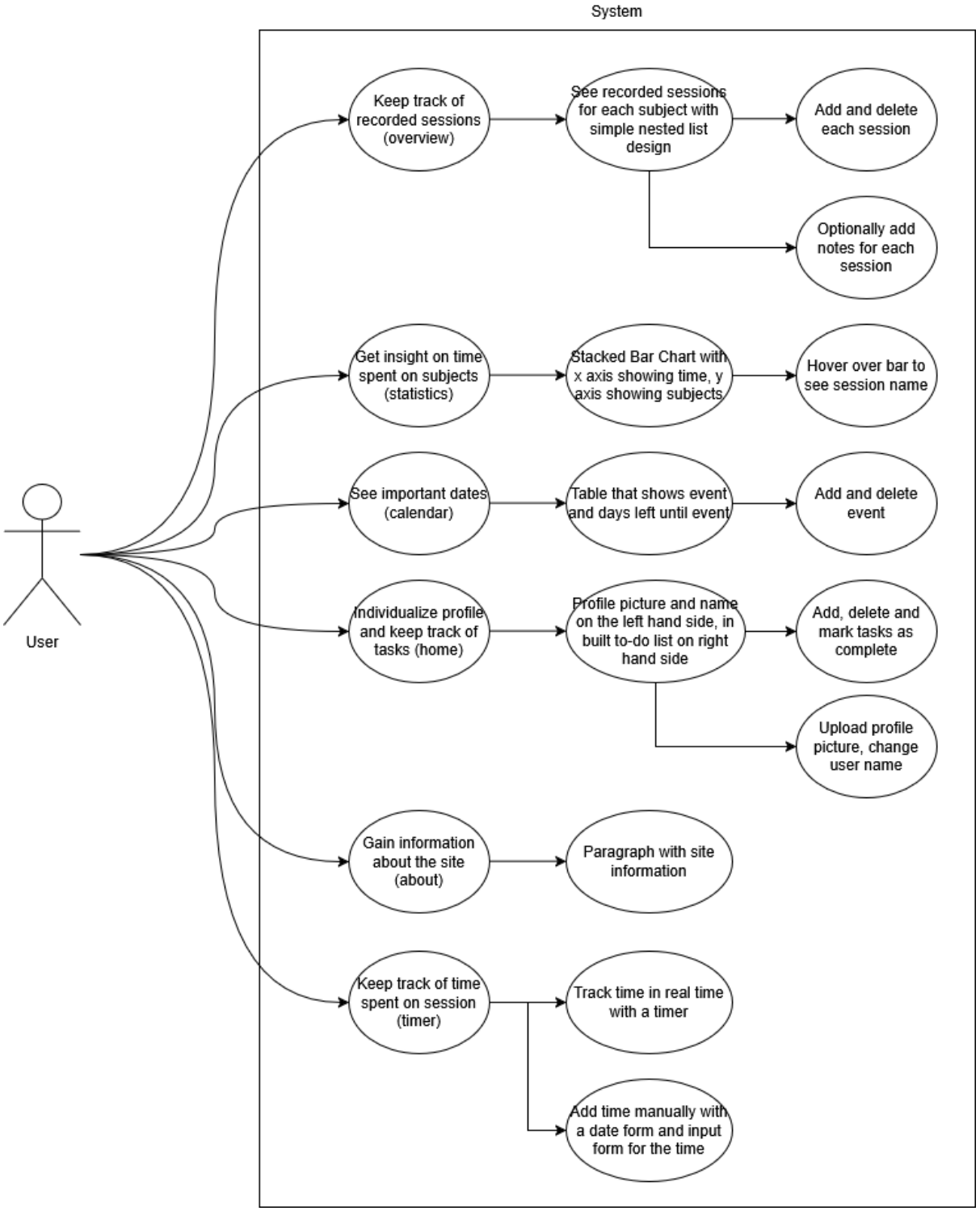
Sitemap

These are all my pages and their functionality.



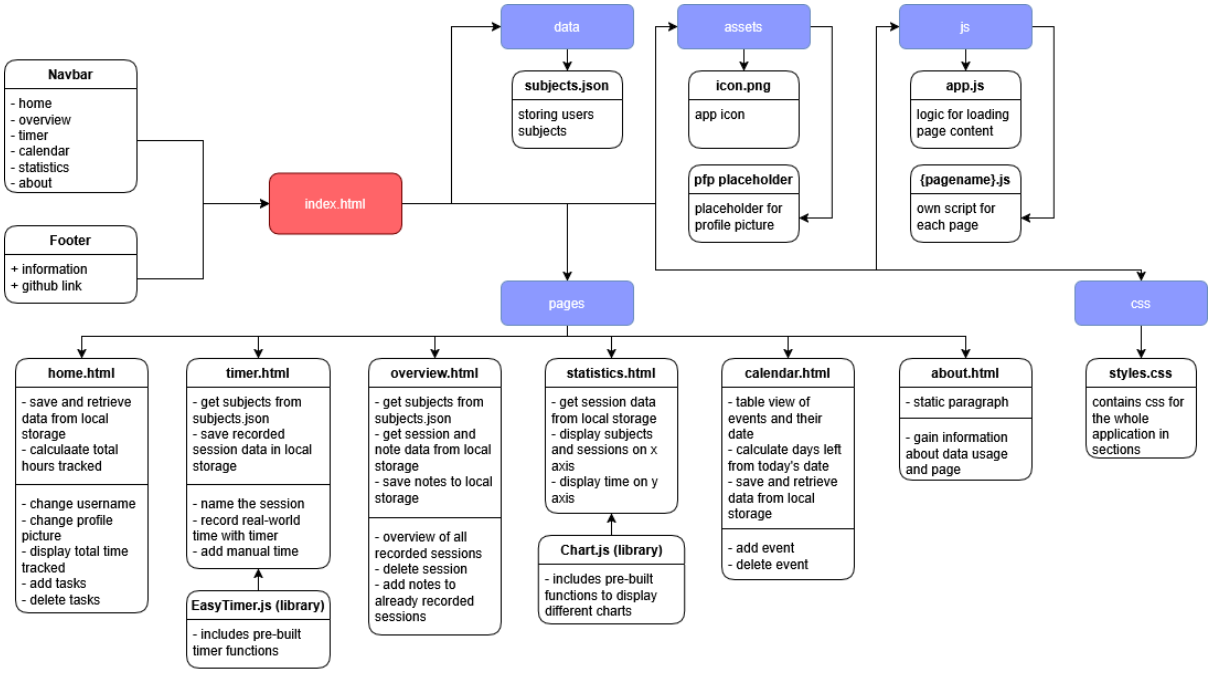
User stories

For each of my functional requirements, I created a user story.



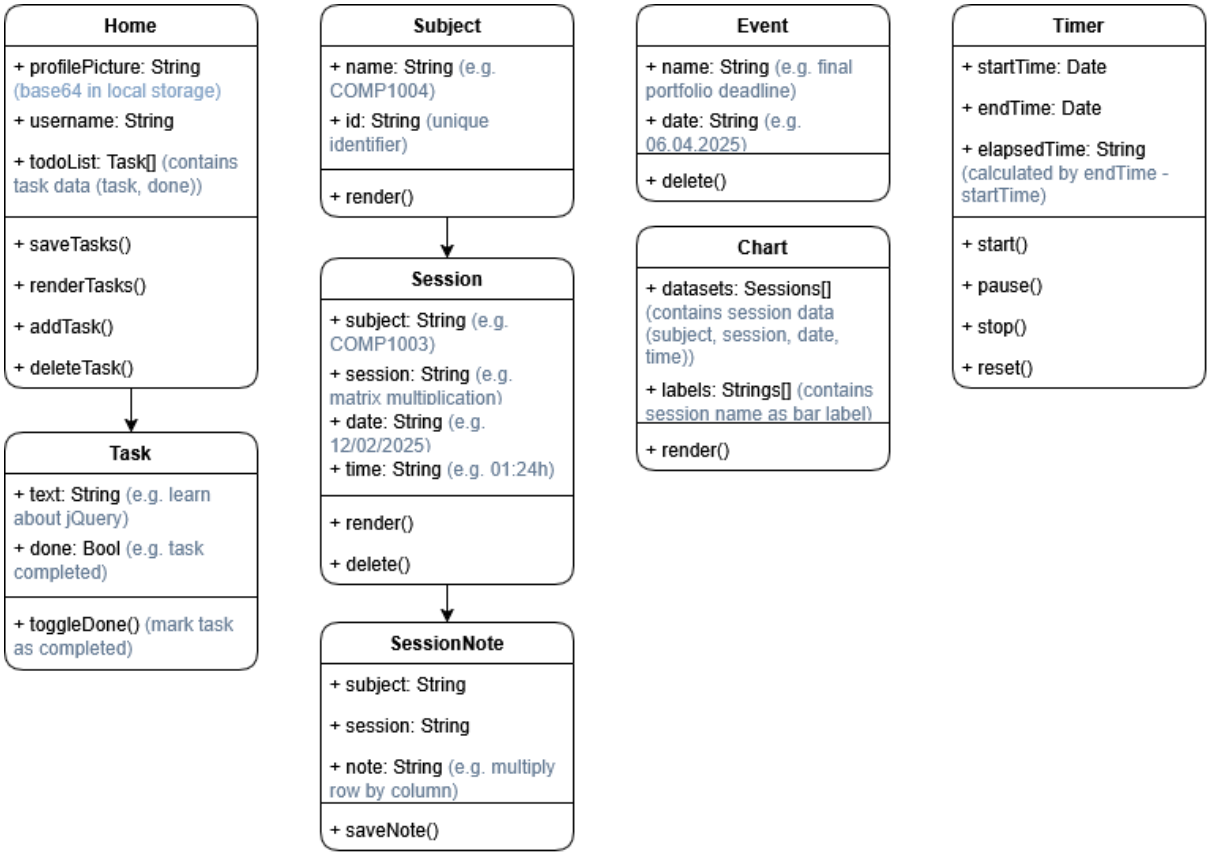
Architecture

As we are developing a single page application, I created an index.html file that contains the navbar and the footer which are the basic layout that is applied to every other .html page in /pages. This is also the root of the application where the live server is started at. The routing for the pages is managed by app.js, which is linked in index.html. The pages folder contains the HTML code for each individual page. Each page also has an accompanying jQuery script in the /js folder, the filename being the same as the .html name, just followed by .js. The /css folder contains all the sites CSS code in styles.css.



Class diagrams

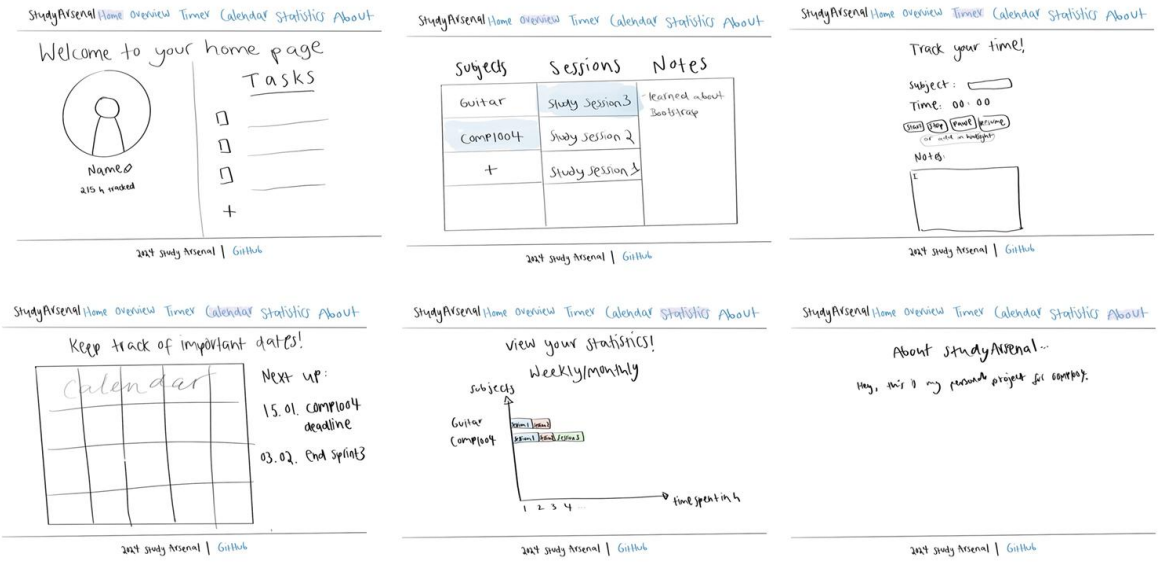
This diagram contains the different classes in each jQuery script.



Rendering updates the displayed data on the page, which is vital for dynamic interaction and responsiveness, therefore it is included in almost every function.

Wireframes

This contains the initial prototype of all my pages, drawn on my iPad using Procreate.



Test cases

For the test cases, I went through each of my pages and tested all the functionalities and noted them down in an Excel spreadsheet.

Test no.	Test summary	Test input	Input type	Expected output	Actual output	Pass/fail
1a	Editing user name	Put in desired name	Normal	Accept any form of text, emojis, languages etc.		
1b	Editing profile picture	Opens folder to select a picture	Normal	Accept only image type		
1c	Adding task to to-do list	Put in desired task name	Normal	Accept any form of text, emojis, languages etc., expand site when needed vertically		
1d	Marking task as completed	Click on task	Normal	Mark task as completed and apply line-through CSS style		
1e	Deleting task	Click on delete button next to task	Normal	Remove task		
1f	Delete all data button	Click on button	Normal	Deletes all local storage		
2a	Display subjects from subjects.json	Define any number of id and name pairs in subjects.json	Normal	Displays subjects from JSON file in subjects tab		
2b	Clicking on a subject	Add sessions through timer page	Normal	Displays sessions for each subject		
2c	Clicking on a session	Click on previously defined session	Normal	Displays the unique notes from that session		
2d	Delete session	Clicking on delete button next to session name	Normal	Deletes this session along with its data		
2e	Edit session notes	Put in desired text	Normal	Accept any form of text, save note for this unique session, expands site with large amount of text		
2f	Export data	Click on button	Normal	Creates a .json file with the user data from local storage		
2g	Import data	Select .json file that was previously exported to import data	Normal	Update the local storage with the key value pairs, duplicates are automatically handled		
2h	Import data	Select random .json file	Erroneous	Does not update the key value pairs that are not expected		

3a	Choose subject	Click on select option field	Normal	Lets the user select subjects from subjects.json		
3b	Enter session name	Putting in text in any text form	Normal	Saves the session with its unique name		
3c	Enter no session name	Enter no session name	Normal	Saves the session with a default name		
3d	Start timer	Click on button	Normal	Start the timer with seconds		
3e	Pause timer	Click on button	Normal	Pause the timer		
3f	Stop timer	Click on button	Normal	Stop the timer, save the session data, alert the user of the saved session and reset the timer		
3g	Reset timer	Click on button	Normal	Reset the timer to 00:00:00		
3h	Manually add date	Click on date selection and select date	Normal	Save the selected date		
3i	Manually add date	Select no date	Normal	Save study session with default date of today		
3j	Manually add study time	Put numbers in hours:minutes format in the text field	Normal	Save the study time		
3k	Manually add study time	Put random text or unformatted numbers in the text field	Erroneous	Inform the user of invalid time format		
3l	Save the manual study time	Click on button	Normal	Saves the session data		
4a	Display statistics	Click on statistics tab in nav bar	Normal	Shows the graph chart with subjects, sessions and hours spent		
5a	Add event name	Putting in text in any text form	Normal	Saves the event with name from text input		
5b	Select event date	Selecting a date, can be in the past or in the future	Normal	Saves the event with date from date input		
5c	Do not select event name or event date	Leave one of the fields empty	Erroneous	Alert the user that both values need to be selected		
5d	Add event	Click on button	Normal	Puts the event in table and displays the days left until the event (- when event is in the past		
5e	Delete event	Click on delete button in delete tab	Normal	Deletes the event		

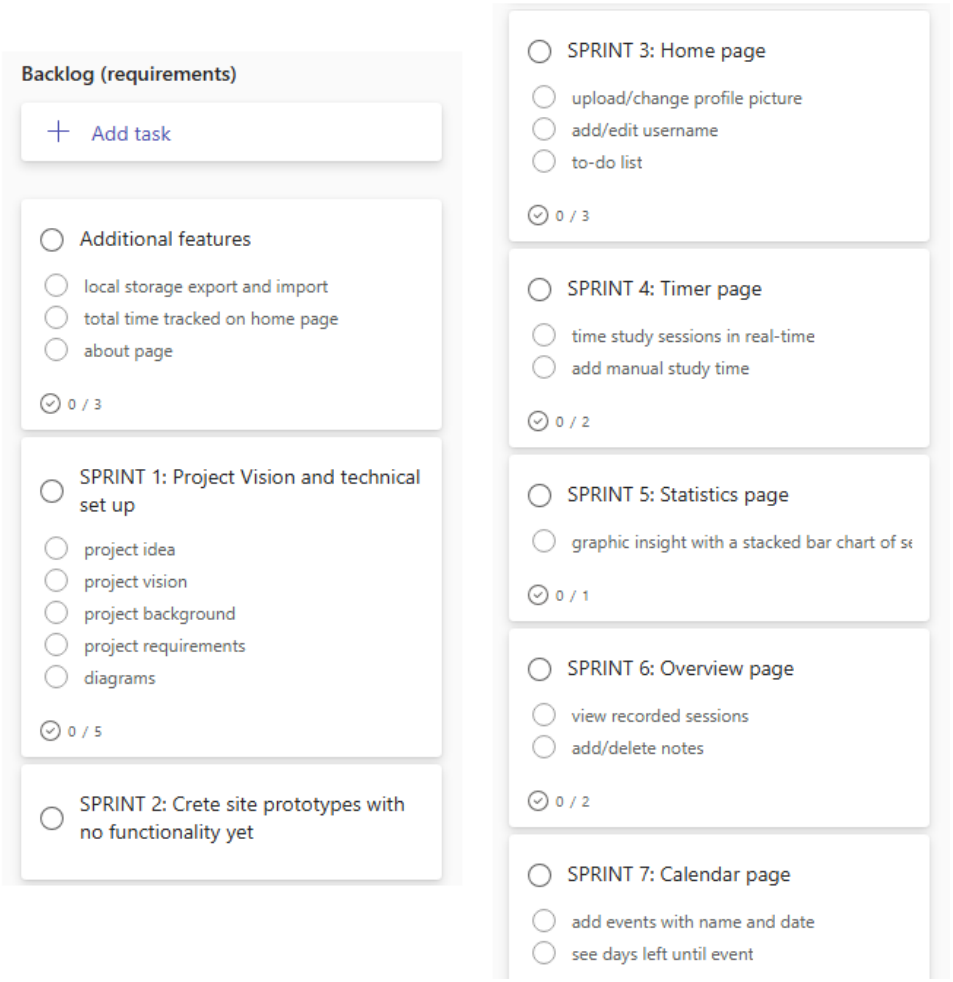
Project Plan Representation

Sprint Planning

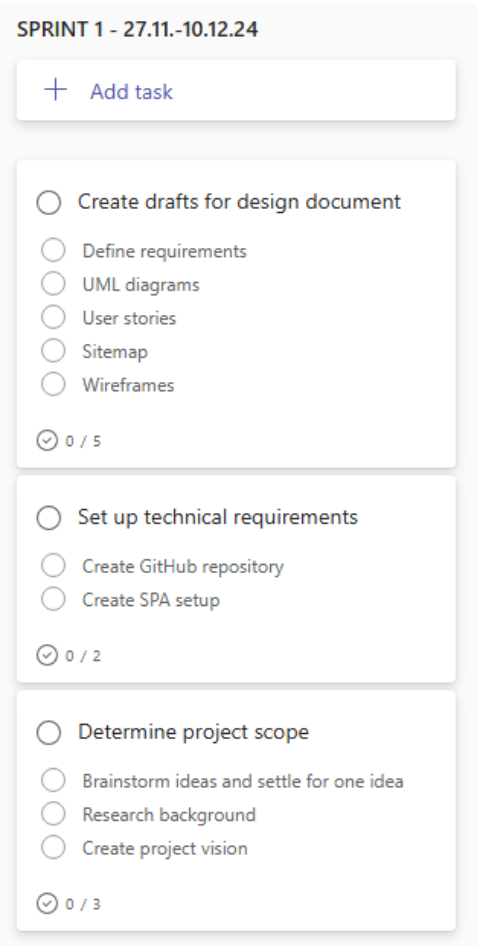
For this module, we were assigned to develop our application in biweekly sprints using Scrum, a framework within the Agile methodology that allows the adjustment of requirements and project scope (Schwaber and Sutherland, 2020).

To keep track of my sprints, I used Microsoft Planner which allowed me to freely plan my application through being able to drag different tasks around. I kept two sections next to my sprints: the backlog and an uncompleted section, containing unfinished tasks. By keeping these two sections next to the active sprint, I could dynamically adapt to changes while still being able to focus on the current sprint. As the development of my application advanced, I also started using its functionalities to organize my tasks.

Firstly, I briefly outlined all the sprints and their objective in the backlog.

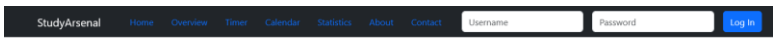


Sprint 1 (27.11.-10.12.24)

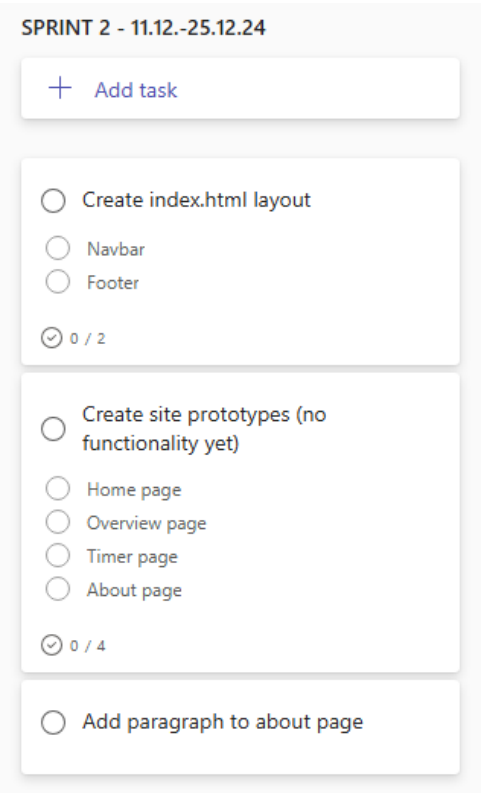


For the first sprint, I focused on defining the project vision, setting up the technical requirements and starting the designing phase, which I successfully accomplished, although my diagrams kept changing until the last sprint as requirements changed.

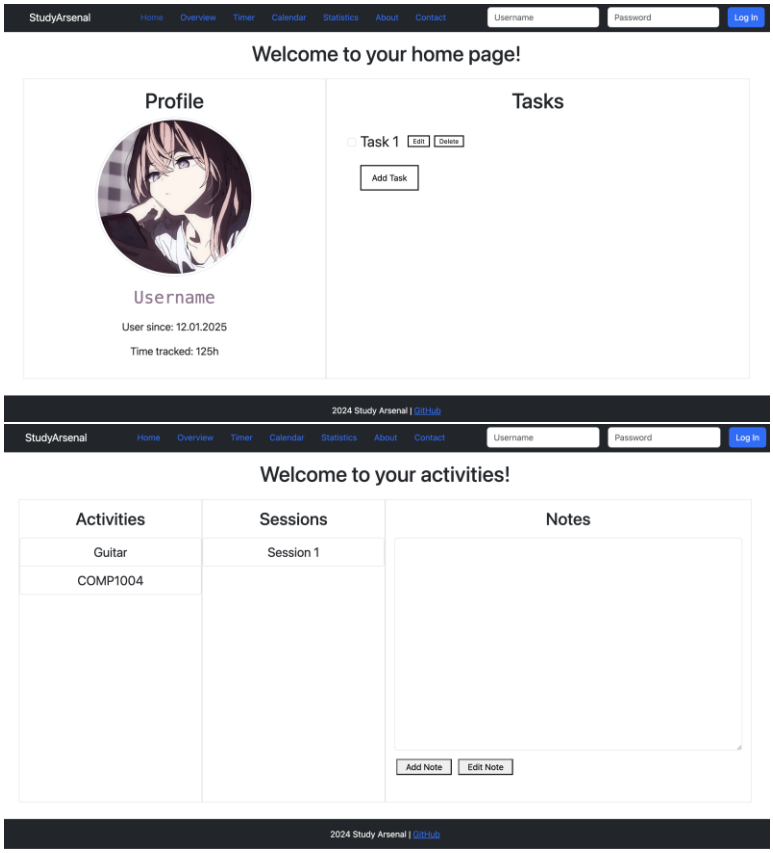
Additionally, I implemented a basic layout for my page in the index.html, including the navbar and footer.



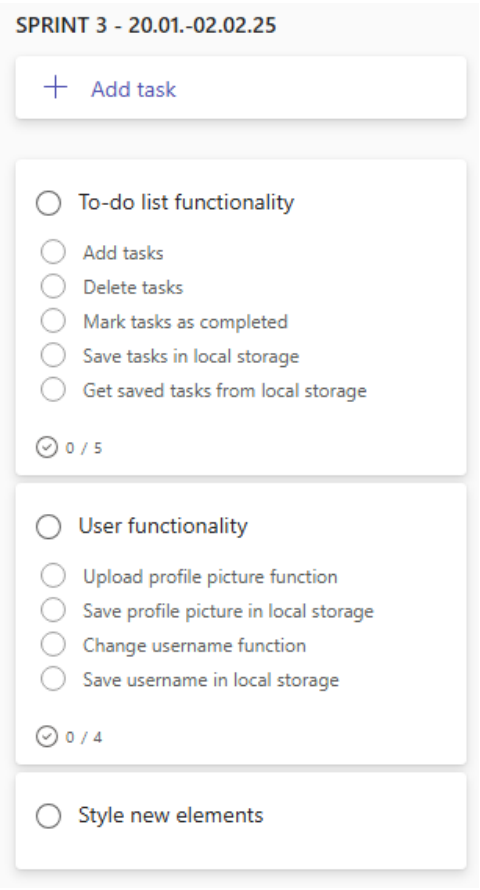
Sprint 2 (11.12.-25.12.24)



In this sprint, I created some page layouts using Bootstrap. I used the Bootstrap original documentation and made use of the grid system and pre-written classes.

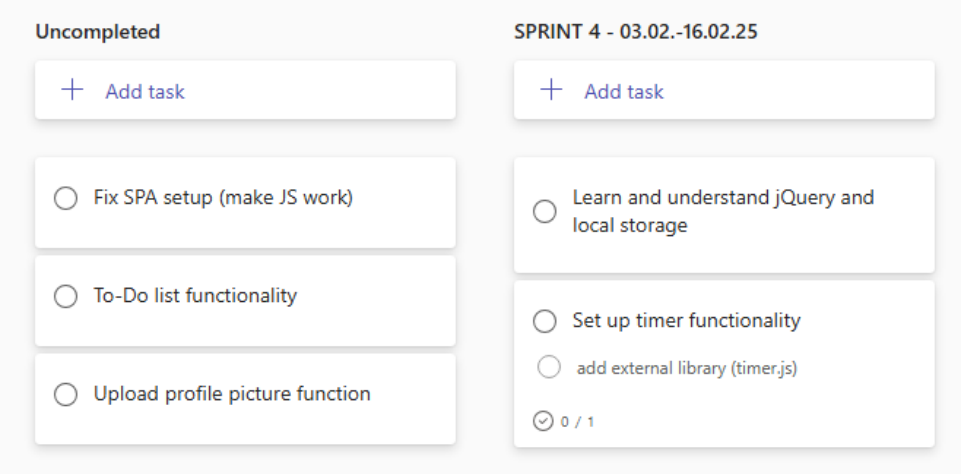


Sprint 3 (20.01.-02.02.25)



The goal for this sprint was to start implementing JS onto my application. However, I was finding myself stuck due the lack of knowledge, which forced me to shift my focus on studying jQuery syntax and local storage functions such as JSON stringify and parse. In addition, I figured my application would not need a user authentication, so I deleted this from my requirements.

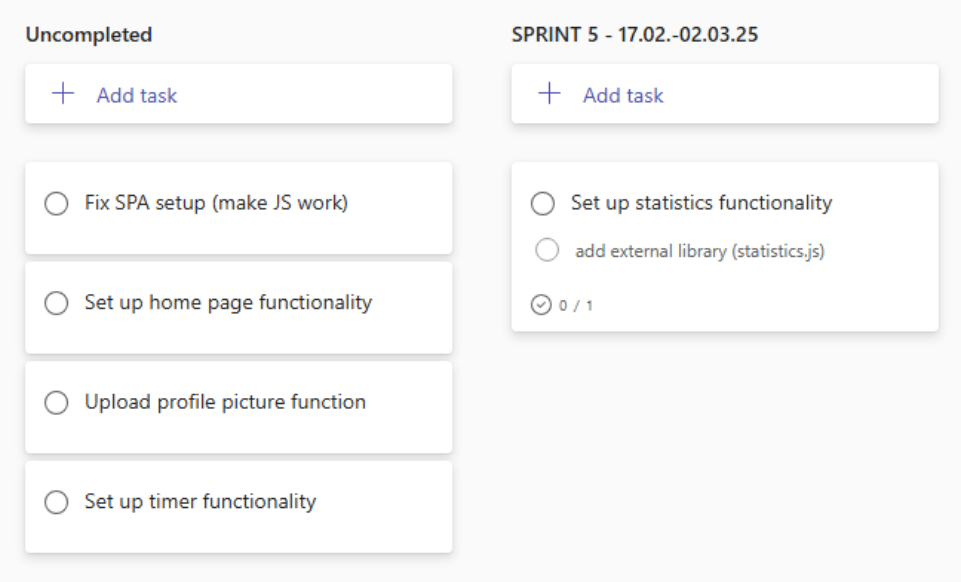
Sprint 4 (03.02.-16.02.25)



This was the first sprint where I had to use the uncompleted bucket, as last sprint left some tasks unfinished. However, as I found myself to be still struggling with the JS, the features I implemented this sprint were very limited and not ideal. I tried using external references, but I was struggling to adjust the code to my desired functionality.

Furthermore, I was struggling with the SPA set up, which had a very high priority of fixing because the scripts for the different pages were not working.

Sprint 5 (17.02.-02.03.25)



In this sprint, I felt more confident in my skills and finally got to implement the functionalities of the last two sprints.

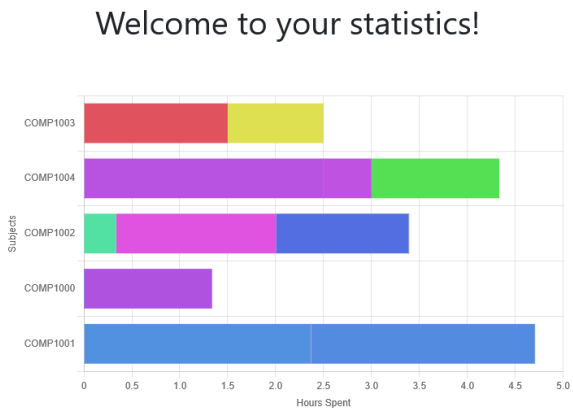
Firstly, I cleaned up the SPA set up (app.js), changing it to a much simpler structure. I used the SPA setup from the PDF on DLE, combining it with my application's needs.

```
wwwroot > js > JS app.js > ...
$(document).ready(function () {
    loadPage('home'); // Load home page by default

    // routing spa set up to load each page when corresponding element with id is clicked (navbar)
    $('#home').click(function () { loadPage('home'); });
    $('#overview').click(function () { loadPage('overview'); });
    $('#timer').click(function () { loadPage('timer'); });
    $('#statistics').click(function () { loadPage('statistics'); });
    $('#calendar').click(function () { loadPage('calendar'); });
    $('#about').click(function () { loadPage('about'); });

    // function to load page
    function loadPage(pageName) {
        $('#page-content-wrapper').load(`pages/${pageName}.html`, function () { // Load the page from /pages
            $.getScript(`js/${pageName}.js`); // Load the js script along with the page from /js
        });
    }
});

```

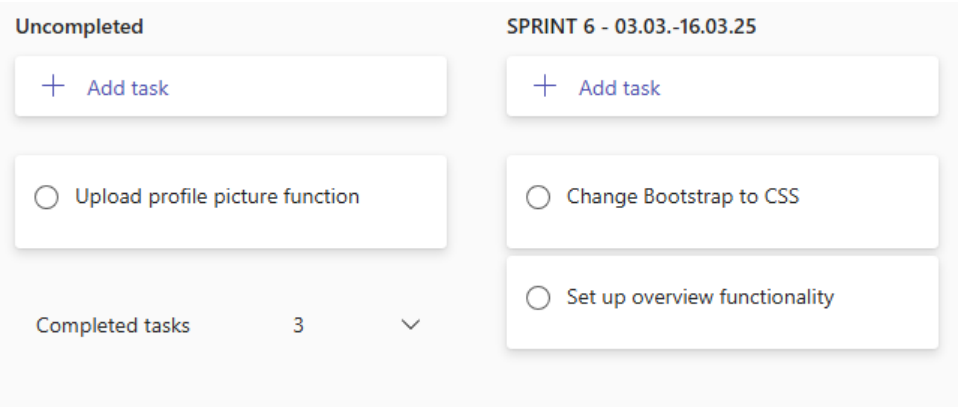
Moreover, having improved my JS skills, I could implement the scripts for the different pages.

For the timer, I used the EasyTimer.js library which provided me with pre-written scripts for the timer functions. I had to adjust the stop button to store the session data in the local storage as an array with the attributes of subject, session, date and time. Moreover, I implemented a manual time add function.

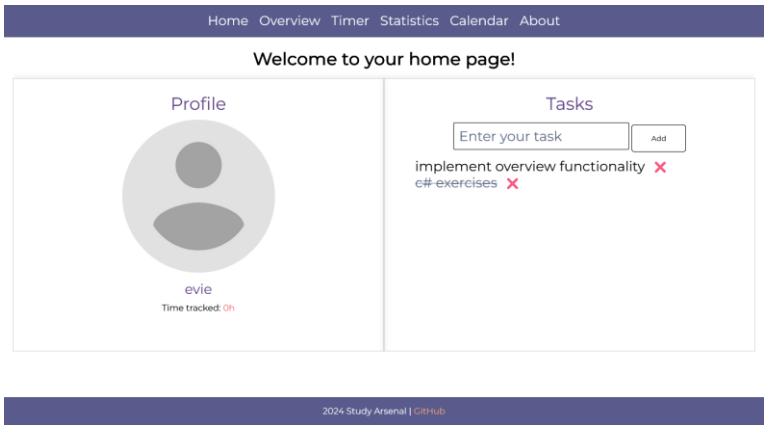
For the statistics page, I used the Chart.js library to display the previously stored session data from the timer page through local storage.



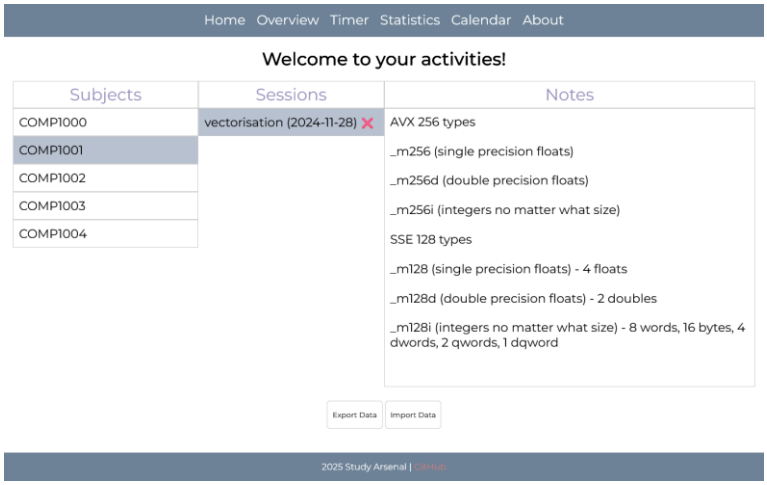
Sprint 6 (03.03.-16.03.25)



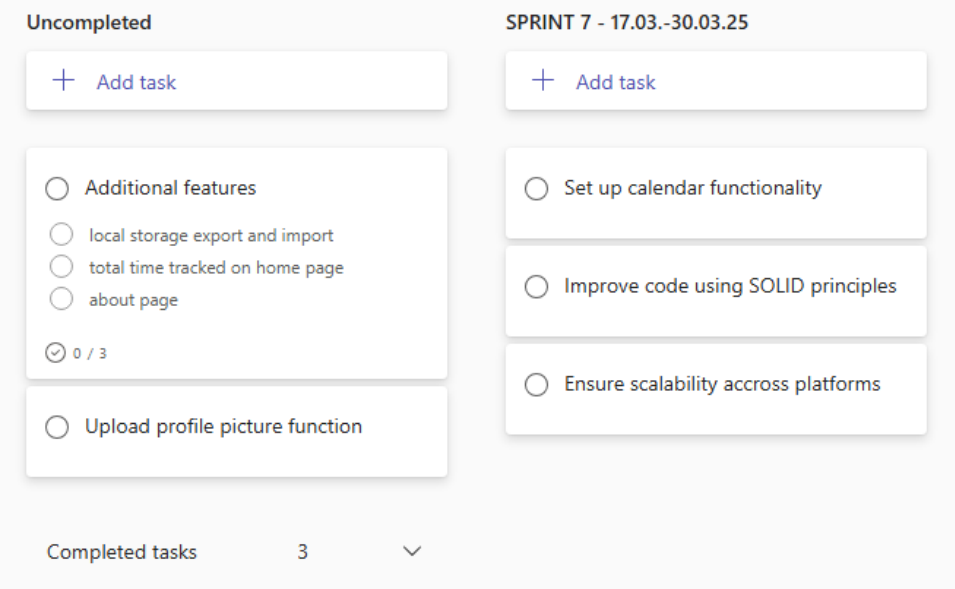
In this week, I found out that we were not allowed to use Bootstrap, therefore I had to focus on changing all my Bootstrap code to CSS code.



Thereafter, I started working to implement the overview page functionality. As this page would be used to display the user's session data, it had to parse the locally stored data and display it. I created two lists to display the subjects and the sessions that belonged to each subject which would be shown when the corresponding subject is clicked. Then, I added a text area where the note for the session could be stored in.



Sprint 7 (17.03.-30.03.25)



For the last sprint, the objective was to develop the calendar. For this, I worked with a table which required me to learn new syntax. I could re-use elements from previous pages, such as the input text and date, so this was done quickly.

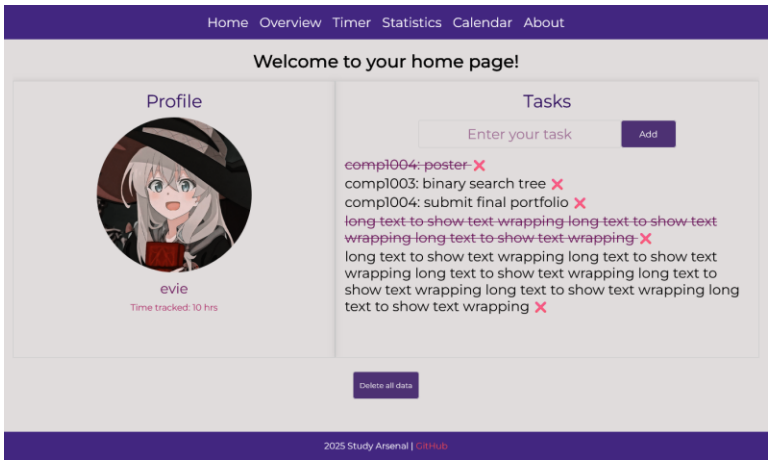
Event Name	Days Until	Delete
final portfolio	10 days	❌
london trip	12 days	❌
final presentation	42 days	❌

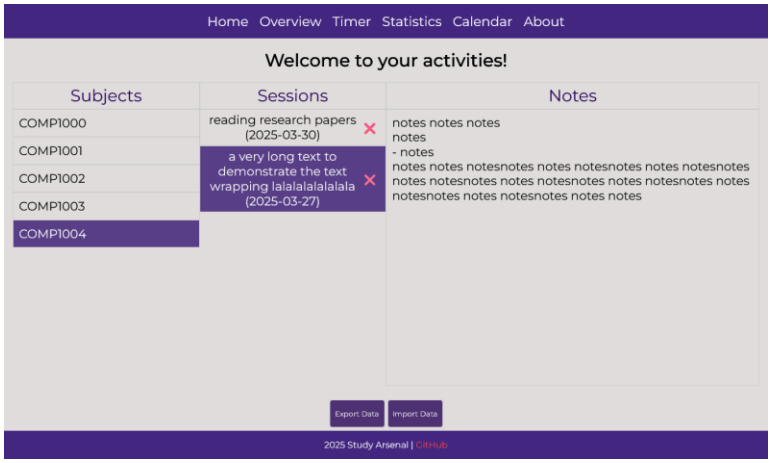
Thereafter, I implemented the additional features that I did not come to develop yet, such as the import and export function.

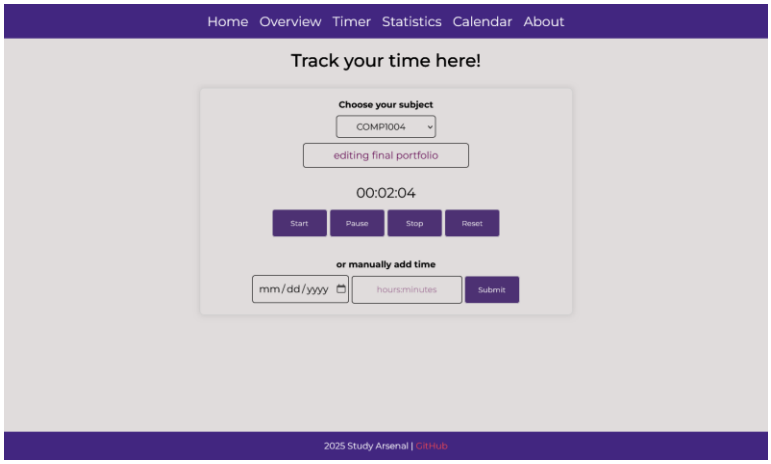
Moreover, I adjusted many CSS styles and worked on improving the code across the whole application.

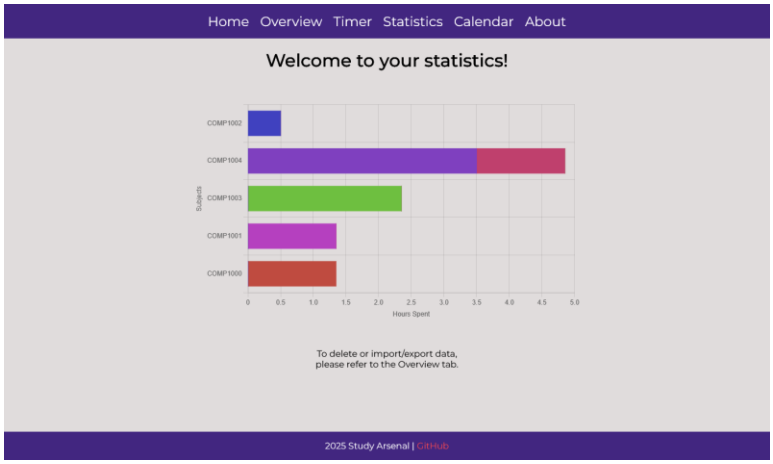
Finally, I tested my application, as provided in the Design Document.

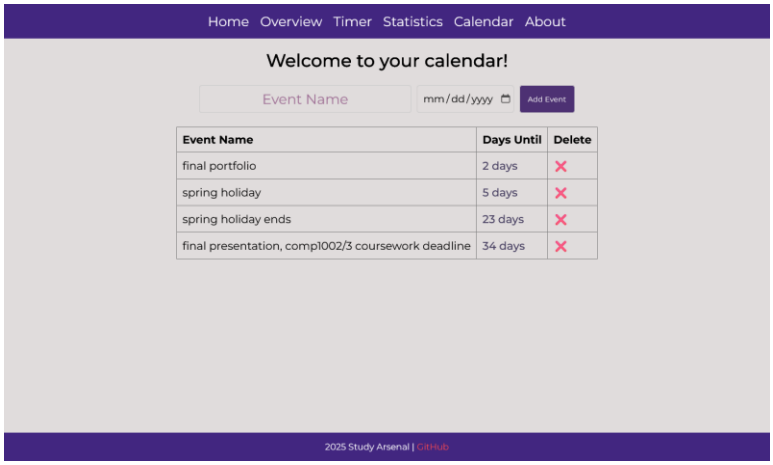
This was how the pages finally looked like.

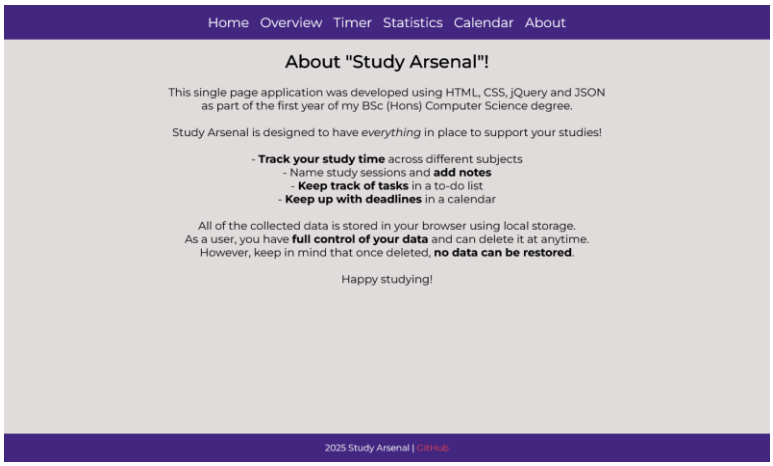












Noted issues and constraints

Physical system

The application should be responsive to ensure scaling across different devices and screen sizes. However, my application currently does not scale well on small devices, limiting its use to desktop devices. Another key limitation is the data storage: my application relies on local storage, which can hold 5-10 MB of data. This however allows to store over 2.5 million words, which is more than enough, as the stored data is mostly text.

Cyber Security

Local storage has many security risks, as it is accessible to any script, meaning that anyone who inserts a script on the page has access to information. Therefore, sensitive information should not be stored on the page. Additionally, the data is not encrypted, making it easily readable to anyone with access. To improve security, data encryption and access control are practices that should be implemented (Matharu, 2024).

Network

The application works completely offline, making it independent from any network connections. However, this comes with the constraint of not being able to sync data across different devices expect using the manual export and import functions. Moreover, data is at risk of being permanently lost if the exported file is deleted or not backed up.

Legal issues

Firstly, I must follow the UK GDPR to protect the user data stored in my application. This law states that the data must be "used fairly, lawfully and transparently [...] limited to only what is necessary" (Government UK, 2024). This is implemented in my application through:

- letting the user know where and how their data is stored (About page)
- limiting data collection by not requesting any additional data (such as email addresses or passwords)
- providing delete buttons for the user to manage their data
- allowing the user to export and import their data

Additionally, any external used libraries must be referenced, which I do in both within my code and in this document under the References section.

Social issues

The application is designed to be accessible to every student. To achieve this, I have avoided complex animations, that might interfere with lower-end devices and accessibility tools and kept a simple user interface.

Moreover, although the application aims to support students in their studies with beneficial outcomes, it may not be suitable for everyone, as study preferences and learning styles differ. For example, some students prefer physical media over digital. Furthermore, students that typically procrastinate on their work will have less anxiety-reducing benefits, as they struggle to engage in active learning that can be tracked (Lay and Schouwenburg, 1993, cited in Adams and Blair, 2019).

Ethical issues

Since my application stores a user's study-related data, a key ethical consideration is how appropriate it is to collect the data. If the data was to be shared and analysed, it could lead to misjudgement on the user's intelligence or unfair comparisons between students, based on their study habits. However, these are not necessarily indicators, as study styles differ. Therefore, if the data is misinterpreted, it could lead to biased conclusions, which are unethical.

Professional issues

To maintain professionality throughout my project, I have used version control to document changes, maintained common coding practices and commented my code. Moreover, I referenced external libraries and other sources.

Reflection

I created a Single-Page-Application that allows the user to manage study relevant organisational tasks and track their time, whereas all the recorded data is stored as local storage, ensuring it doesn't get lost when reloading the page or application. Moreover, the data can be exported and imported, ensuring usage across different browsers and devices.

First, I must address that throughout my project, I have met several organisational issues related to the module. Some requirements changed, such as using Bootstrap and the requirements were poorly defined overall, which made me struggle to understand what exactly was expected from me. For example, the provided sample portfolios were very different from each other, which resulted in confusion on what to include in my final portfolio (which is why some parts of this portfolio might be redundant or missing...). The template.docx was also missing many sections that were included in the marking scheme or sample portfolios.

Working on the application itself, implementing Scrum made me have a clear direction to go and I found it very effective to work with. Although for some sprints I fell behind, for other sprints, I implemented more than planned, which balanced it out. Additionally, Scrum allowed me to change requirements, allowing for dynamic adjustments. However, it was only in the last sprint that I implemented very important features, such as the data export and import feature as they were defined in my "additional features" bucket, which made me feel pressured due to the little time left. In the future, I would assign them into a sprint instead of leaving them open-ended.

The coding process was difficult, especially at the beginning, since I had to teach many of the coding requirements myself (HTML, CSS, jQuery and JSON), but over the course of the project, my understanding significantly improved. While developing my code, I constantly used the internet to find information on platforms such as YouTube, Stack Overflow and w3schools. However, combined with the learning curve, this resulted in an inconsistent coding style across the application and subsequently lead messy code that must be cleaned up in retrospect.

There were some features that I did not finish to implement, which include the ability for the user to change their saved data in retrospect, allowing to add labels in the to-do list and adding weekly and monthly views for the statistics. For the future, I would also consider integrating features that improve user engagement, such as achievements badges, colour themes, day streaks and habit logs. Moreover, if the application was to be scaled, it could be connected to a database and hosted on a web server.

Overall, I obtained valuable project planning and technical skills through working on this project and built a foundation to understand modern concepts like React. Moreover, it taught me how to resolve obstacles that felt impossible to overcome at first. Most significantly however is the practical use of the application for me and potentially other interested students, rather than merely serving as a mandatory university coursework.

GitHub repository

https://github.com/cryingongit/StudyArsenal

References

Adams, R. V. & Blair, E. (2019) 'Impact of Time Management Behaviors on Undergraduate Engineering Students' Performance', *SAGE Open, 9(1)*. Available at: https://doi.org/10.1177/2158244018824506.

GeekForGeeks (2025) 'Software Development Life Cycle (SDLC)', *GeeksForGeeks*. Available at: https://www.geeksforgeeks.org/software-development-life-cycle-sdlc/ (Accessed: 28 March 2025).

Government UK (n.d.) 'Data protection'. *Gov.uk*. Available at: https://www.gov.uk/data-protection (Accessed: 23 March 2025).

Hockings, C. *et al.* (2017) 'Independent learning – what we do when you're not there', *Teaching in Higher Education,* 23(2), pp. 145–161. Available at: https://doi.org/10.1080/13562517.2017.1332031.

Matharu, M. (2024) '*When Not to Use Local Storage: Risks, Examples, and Secure Alternatives'*, *Medium*. Available at: https://meenumatharu.medium.com/when-not-to-use-local-storage-risks-examples-and-secure-alternatives-de541fed56d2 (Accessed: 23 March 2025).

Schwaber, K. & Sutherland, J. (2020) *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Available at: https://scrumguides.org (Accessed: 3 March 2025).

Svartdal, F., Dahl, T. I., Gamst-Klaussen, T., Koppenborg, M. and Klingsieck, K. B. (2020) 'How Study Environments Foster Academic Procrastination: Overview and Recommendations', *Frontiers in Psychology,* 11. Available at: https://doi.org/10.3389/fpsyg.2020.540910 .

External libraries used for the application

https://jquery.com/

https://albert-gonzalez.github.io/easytimer.js/

https://www.chartjs.org/